

[Gemini Deep Research \(/gemini-api/docs/deep-research\)](/gemini-api/docs/deep-research) is now available in preview with collaborative planning, visualization, MCP support, and more.

Gemini Deep Research Agent

The Gemini Deep Research Agent autonomously plans, executes, and synthesizes multi-step research tasks. Powered by Gemini, it navigates complex information landscapes to produce detailed, cited reports. New capabilities allow you to collaboratively plan with the agent, connect to external tools using MCP servers, include visualizations (like charts and graphs), and provide documents directly as input.

Research tasks involve iterative searching and reading and can take several minutes to complete. You must use background execution (set `background=true`) to run the agent asynchronously and poll for results or stream updates. See [Handling long running tasks \(#long-running-tasks\)](#) for more details.

Preview: The Gemini Deep Research Agent is currently in preview. The Deep Research agent is exclusively available using the [Interactions API \(/gemini-api/docs/interactions\)](/gemini-api/docs/interactions). You cannot access it through `generate_content`.

The following example shows how to start a research task in the background and poll for results.

PythonJavaScript (#javascript)**REST** (#rest)
(#python)

```
import time
from google import genai

client = genai.Client()

interaction = client.interactions.create(
    input="Research the history of Google TPUs.",
    agent="deep-research-preview-04-2026",
    background=True,
)

print(f"Research started: {interaction.id}")
```

```
while True:
    interaction = client.interactions.get(interaction.id)
    if interaction.status == "completed":
        print(interaction.output_text)
        break
    elif interaction.status == "failed":
        print(f"Research failed: {interaction.error}")
        break
    time.sleep(10)
```

Supported Versions

The Deep Research agent comes in two versions:

- **Deep Research** (deep-research-preview-04-2026): Designed for speed and efficiency, ideal to be streamed back to a client UI.
- **Deep Research Max** (deep-research-max-preview-04-2026): Maximum comprehensiveness for automated context gathering and synthesis.

Collaborative planning

Preview: Collaborative planning allows you to review and refine the research plan before execution.

Collaborative planning gives you control over the research direction before the agent starts its work. When enabled, the agent returns a proposed research plan instead of executing immediately. You can then review, modify, or approve the plan through multi-turn interactions.

Step 1: Request a plan

Set `collaborative_planning=True` in the first interaction. The agent returns a research plan instead of a full report.

PythonJavaScript (#javascript)REST (#rest)
(#python)

```

from google import genai

client = genai.Client()

plan_interaction = client.interactions.create(
    agent="deep-research-preview-04-2026",
    input="Do some research on Google TPUs.",
    agent_config={
        "type": "deep-research",
        "thinking_summaries": "auto",
        "collaborative_planning": True,
    },
    background=True,
)

while (result := client.interactions.get(id=plan_interaction.id)).status
    time.sleep(5)
print(result.output_text)

```

Step 2: Refine the plan (optional)

Use `previous_interaction_id` to continue the conversation and iterate on the plan. Keep `collaborative_planning=True` to stay in planning mode.

PythonJavaScript (#javascript)REST (#rest)
(#python)

```

refined_plan = client.interactions.create(
    agent="deep-research-preview-04-2026",
    input="Focus more on the differences between Google TPUs and competi
    agent_config={
        "type": "deep-research",
        "thinking_summaries": "auto",
        "collaborative_planning": True,
    },
    previous_interaction_id=plan_interaction.id,
    background=True,
)

while (result := client.interactions.get(id=refined_plan.id)).status !=
    time.sleep(5)
print(result.output_text)

```

Step 3: Approve and execute

Set `collaborative_planning=False` (or omit it) to approve the plan and start the research.

PythonJavaScript (#javascript)REST (#rest)
(#python)

```
final_report = client.interactions.create(
    agent="deep-research-preview-04-2026",
    input="Plan looks good!",
    agent_config={
        "type": "deep-research",
        "thinking_summaries": "auto",
        "collaborative_planning": False,
    },
    previous_interaction_id=refined_plan.id,
    background=True,
)

while (result := client.interactions.get(id=final_report.id)).status !=
    time.sleep(5)
print(result.output_text)
```

Visualization

Preview: Visualization allows the agent to generate charts and graphs to support its findings.

When `visualization` is set to `"auto"`, the agent can generate charts, graphs, and other visual elements to support its research findings. Generated images are included in the response steps and streamed as `image` deltas. For best results, explicitly ask for visuals in your query — for example, "Include charts showing trends over time" or "Generate graphics comparing market share." Setting `visualization` to `"auto"` enables the capability, but the agent generates visuals only when the prompt requests them.

PythonJavaScript (#javascript)REST (#rest)
(#python)

```
import base64

interaction = client.interactions.create(
```

```
agent="deep-research-preview-04-2026",
input="Analyze global semiconductor market trends. Include graphics
agent_config={
    "type": "deep-research",
    "visualization": "auto",
},
background=True,
)

print(f"Research started: {interaction.id}")

while (result := client.interactions.get(id=interaction.id)).status != "
    time.sleep(5)

for step in result.steps:
    if step.type == "model_output":
        for content_item in step.content:
            if content_item.type == "text":
                print(content_item.text)
            elif content_item.type == "image" and content_item.data:
                image_bytes = base64.b64decode(content_item.data)
                print(f"Received image: {len(image_bytes)} bytes")
```

Supported tools

Deep Research supports multiple built-in and external tools. By default (when no `tools` parameter is provided), the agent has access to Google Search, URL Context, and Code Execution. You can explicitly specify tools to restrict or extend the agent's capabilities.

Tool	Type value	Description
Google Search	<code>google_search</code>	Search the public web. Enabled by default.
URL Context	<code>url_context</code>	Read and summarize web page content. Enabled by default.
Code Execution	<code>code_execution</code>	Execute code to perform calculations and data analysis. Enabled by default.
MCP Server	<code>mcp_server</code>	Connect to remote MCP servers for external tool access.

Tool	Type value	Description
File Search	<code>file_search</code>	Search your uploaded document corpora.

Google Search

Explicitly enable Google Search as the only tool:

PythonJavaScript (#javascript)REST (#rest)
(#python)

```
interaction = client.interactions.create(
    agent="deep-research-preview-04-2026",
    input="What are the latest developments in quantum computing?",
    tools=[{"type": "google_search"}],
    background=True,
)
```

URL Context

Give the agent the ability to read and summarize specific web pages:

PythonJavaScript (#javascript)REST (#rest)
(#python)

```
interaction = client.interactions.create(
    agent="deep-research-preview-04-2026",
    input="Summarize the content of https://www.wikipedia.org/",
    tools=[{"type": "url_context"}],
    background=True,
)
```

Code Execution

Allow the agent to execute code for calculations and data analysis:

PythonJavaScript (#javascript)REST (#rest)
(#python)

```
interaction = client.interactions.create(
    agent="deep-research-preview-04-2026",
    input="Calculate the 50th Fibonacci number.",
    tools=[{"type": "code_execution"}],
    background=True,
)
```

MCP servers

Preview: Connect to remote MCP servers to give the agent access to external tools and services.

Provide the server name and url in the tools configuration. You can also pass authentication credentials and restrict which tools the agent can call.

Field	Type	Required	Description
type	string	Yes	Must be "mcp_server".
name	string	No	A display name for the MCP server.
url	string	No	The full URL for the MCP server endpoint.
headers	object	No	Key-value pairs sent as HTTP headers with every request to the server (for example, authentication tokens).
allowed_tools	array	No	Restrict which tools from the server the agent may call.

Basic usage

PythonJavaScript (#javascript)REST (#rest)
(#python)

```
interaction = client.interactions.create(
    agent="deep-research-preview-04-2026",
    input="Check the status of my last server deployment.",
    tools=[
```

```
{
  "type": "mcp_server",
  "name": "Deployment Tracker",
  "url": "https://mcp.example.com/mcp",
  "headers": {"Authorization": "Bearer my-token"},
}
],
background=True,
)
```

File Search

Give the agent access to your own data by using the [File Search](#) (/gemini-api/docs/interactions/file-search) tool.

PythonJavaScript (#javascript)**REST** (#rest)
(#python)

```
import time
from google import genai

client = genai.Client()

interaction = client.interactions.create(
    input="Compare our 2025 fiscal year report against current public we
    agent="deep-research-preview-04-2026",
    background=True,
    tools=[
        {
            "type": "file_search",
            "file_search_store_names": ['fileSearchStores/my-store-name']
        }
    ]
)
```

Steerability and formatting

You can steer the agent's output by providing specific formatting instructions in your prompt. This allows you to structure reports into specific sections and subsections, include data tables, or adjust tone for different audiences (e.g., "technical," "executive," "casual").

Define the desired output format explicitly in your input text.

PythonJavaScript (#javascript)REST (#rest)
(#python)

```
prompt = """
Research the competitive landscape of EV batteries.

Format the output as a technical report with the following structure:
1. Executive Summary
2. Key Players (Must include a data table comparing capacity and chemist
3. Supply Chain Risks
"""

interaction = client.interactions.create(
    input=prompt,
    agent="deep-research-preview-04-2026",
    background=True
)
```

Multimodal inputs

Deep Research supports multimodal inputs, including images and documents (PDFs), allowing the agent to analyze visual content and conduct web-based research contextualized by the provided inputs.

PythonJavaScript (#javascript)REST (#rest)
(#python)

```
import time
from google import genai

client = genai.Client()

prompt = """Analyze the interspecies dynamics and behavioral risks prese
in the provided image of the African watering hole. Specifically, invest
the symbiotic relationship between the avian species and the pachyderms
shown, and conduct a risk assessment for the reticulated giraffes based
their drinking posture relative to the specific predator visible in the
foreground."""

interaction = client.interactions.create(
```

```

input=[
    {"type": "text", "text": prompt},
    {
        "type": "image",
        "mime_type": "image/jpeg",
        "uri": "https://storage.googleapis.com/generativeai-download
    }
],
agent="deep-research-preview-04-2026",
background=True
)

print(f"Research started: {interaction.id}")

while True:
    interaction = client.interactions.get(interaction.id)
    if interaction.status == "completed":
        print(interaction.output_text)
        break
    elif interaction.status == "failed":
        print(f"Research failed: {interaction.error}")
        break
    time.sleep(10)

```

Document understanding

Preview: Document understanding allows passing documents directly as multimodal input.

Pass documents directly as multimodal input. The agent analyzes the provided documents and conducts research grounded in their content.

PythonJavaScript (#javascript)REST (#rest)
(#python)

```

from google import genai

client = genai.Client()

interaction = client.interactions.create(
    agent="deep-research-preview-04-2026",
    input=[
        {"type": "text", "text": "What is this document about?"},
        {

```

```
        "type": "document",
        "uri": "https://arxiv.org/pdf/1706.03762",
        "mime_type": "application/pdf",
    },
],
background=True,
)
```

Handling long-running tasks

Deep Research is a multi-step process involving planning, searching, reading, and writing. This cycle typically exceeds the standard timeout limits of synchronous API calls.

Agents are required to use `background=True`. The API returns a partial `Interaction` object immediately. You can use the `id` property to retrieve an interaction for polling. The interaction state will transition from `in_progress` to `completed` or `failed`.

Streaming

Deep Research supports streaming to receive real-time updates on the research progress including thought summaries, text output, and generated images. You must set `stream=True` and `background=True`. For a comprehensive guide to streaming, including event types, tool streaming, and thinking, see [Streaming interactions](https://ai.google.dev/gemini-api/docs/interactions/streaming) (`/gemini-api/docs/interactions/streaming`).

To receive intermediate reasoning steps (thoughts) and progress updates, you must enable **thinking summaries** by setting `thinking_summaries` to `"auto"` in the `agent_config`. Without this, the stream may only provide the final results.

Note: The streaming connection can drop or expire. Since Deep Research tasks can run longer, your application should check the interaction status and reconnect as shown in the examples below.

Stream event types

Event type	Delta type	Description
<code>step.delta</code>	<code>thought</code>	Intermediate reasoning step from the agent.

Event type	Delta type	Description
step.delta	text	Part of the final text output.
step.delta	image	A generated image (base64-encoded).

The following example starts a research task and processes the stream with automatic reconnection. It tracks the `interaction_id` and `last_event_id` so that if the connection drops (for example, after the 600-second timeout), it can resume from where it left off.

PythonJavaScript (#javascript)REST (#rest)
(#python)

```
from google import genai

client = genai.Client()

interaction_id = None
last_event_id = None
is_complete = False

def process_stream(stream):
    global interaction_id, last_event_id, is_complete
    for event in stream:
        if event.event_type == "interaction.created":
            interaction_id = event.interaction.id
        if event.event_id:
            last_event_id = event.event_id
        if event.event_type == "step.delta":
            if event.delta.type == "text":
                print(event.delta.text, end="", flush=True)
            elif event.delta.type == "thought":
                print(f"Thought: {event.delta.text}", flush=True)
            elif event.event_type in ("interaction.completed", "error"):
                is_complete = True

stream = client.interactions.create(
    input="Research the history of Google TPUs.",
    agent="deep-research-preview-04-2026",
    background=True,
    stream=True,
    agent_config={"type": "deep-research", "thinking_summaries": "auto"}
)
process_stream(stream)
```

```
while not is_complete and interaction_id:
    status = client.interactions.get(interaction_id)
    if status.status != "in_progress":
        break
    stream = client.interactions.get(
        id=interaction_id, stream=True, last_event_id=last_event_id,
    )
    process_stream(stream)
```

Follow-up questions and interactions

You can continue the conversation after the agent returns the final report by using the `previous_interaction_id`. This lets you to ask for clarification, summarization or elaboration on specific sections of the research without restarting the entire task.

PythonJavaScript (#javascript)REST (#rest)
(#python)

```
import time
from google import genai

client = genai.Client()

interaction = client.interactions.create(
    input="Can you elaborate on the second point in the report?",
    model="gemini-3.1-pro-preview",
    previous_interaction_id="COMPLETED_INTERACTION_ID"
)

print(interaction.output_text)
```

When to use Gemini Deep Research Agent

Deep Research is an **agent**, not just a model. It is best suited for workloads that require an "analyst-in-a-box" approach rather than low-latency chat.

Feature Standard Gemini Models		Gemini Deep Research Agent
Latency Seconds		Minutes (Async/Background)
ProcessGenerate -> Output		Plan -> Search -> Read -> Iterate -> Output
Output Conversational text, code, short summaries		Detailed reports, long-form analysis, comparative tables
Best For Chatbots, extraction, creative writing		Market analysis, due diligence, literature reviews, competitive landscaping

Agent configuration

Deep Research uses the `agent_config` parameter to control behavior. Pass it as a dictionary with the following fields:

Field	Type	Default	Description
<code>type</code>	<code>string</code>	Required	Must be <code>"deep-research"</code> .
<code>thinking_summaries</code>	<code>string</code>	<code>"none"</code>	Set to <code>"auto"</code> to receive intermediate reasoning steps during streaming. Set to <code>"none"</code> to disable.
<code>visualization</code>	<code>string</code>	<code>"auto"</code>	Set to <code>"auto"</code> to enable agent-generated charts and images. Set to <code>"off"</code> to disable.
<code>collaborative_planning</code>	<code>boolean</code>	<code>false</code>	Set to <code>true</code> to enable multi-turn plan review before research begins.

PythonJavaScript (#javascript)REST (#rest)
(#python)

```
agent_config = {  
    "type": "deep-research",  
    "thinking_summaries": "auto",  
    "visualization": "auto",  
}
```

```
"collaborative_planning": False,  
}  
  
interaction = client.interactions.create(  
    agent="deep-research-preview-04-2026",  
    input="Research the competitive landscape of cloud GPUs.",  
    agent_config=agent_config,  
    background=True,  
)
```

Availability and pricing

You can access the Gemini Deep Research Agent using the Interactions API in Google AI Studio and the Gemini API.

Pricing follows a [pay-as-you-go model](https://ai.google.dev/gemini-api/docs/pricing#pricing-for-agents) (/gemini-api/docs/pricing#pricing-for-agents) based on the underlying Gemini models and the specific tools the agent utilizes. Unlike standard chat requests, where a request leads to one output, a Deep Research task is an agentic workflow. A single request triggers an autonomous loop of planning, searching, reading, and reasoning.

Estimated costs

Costs vary based on the depth of research required. The agent autonomously determines how much reading and searching is necessary to answer your prompt.

- **Deep Research** (deep-research-preview-04-2026): For a typical query requiring moderate analysis, the agent might use ~80 search queries, ~250k input tokens (~50-70% cached), and ~60k output tokens.
 - **Estimated total:** ~\$1.00 – \$3.00 per task
- **Deep Research Max** (deep-research-max-preview-04-2026): For deep competitive landscape analysis or extensive due diligence, the agent might use up to ~160 search queries, ~900k input tokens (~50-70% cached), and ~80k output tokens.
 - **Estimated total:** ~\$3.00 – \$7.00 per task

Note: These figures are estimates based on preview rates and are subject to change.

Safety considerations

Giving an agent access to the web and your private files requires careful consideration of safety risks.

- **Prompt injection using files:** The agent reads the contents of the files you provide. Ensure that uploaded documents (PDFs, text files) come from trusted sources. A malicious file could contain hidden text designed to manipulate the agent's output.
- **Web content risks:** The agent searches the public web. While we implement robust safety filters, there is a risk that the agent may encounter and process malicious web pages. We recommend reviewing the citations provided in the response to verify the sources.
- **Exfiltration:** Be cautious when asking the agent to summarize sensitive internal data if you are also allowing it to browse the web.

Best practices

- **Prompt for unknowns:** Instruct the agent on how to handle missing data. For example, add *"If specific figures for 2025 are not available, explicitly state they are projections or unavailable rather than estimating"* to your prompt.
- **Provide context:** Ground the agent's research by providing background information or constraints directly in the input prompt.
- **Use collaborative planning:** For complex queries, enable collaborative planning to review and refine the research plan before execution.
- **Multimodal inputs:** Deep Research Agent supports multi-modal inputs. Use cautiously, as this increases costs and risks context window overflow.

Limitations

- **Beta status:** The Interactions API is in public beta. Features and schemas may change.
- **Custom tools:** You cannot currently provide custom Function Calling tools but you can use remote MCP (Model Context Protocol) servers with the Deep Research agent.
- **Structured output:** The Deep Research Agent currently doesn't support structured outputs.

- **Max research time:** The Deep Research agent has a maximum research time of 60 minutes. Most tasks should complete within 20 minutes.
- **Store requirement:** Agent execution using `background=True` requires `store=True`.
- **Google search:** [Google Search](/gemini-api/docs/interactions/google-search) (/gemini-api/docs/interactions/google-search) is enabled by default and [specific restrictions](/gemini-api/terms#use-restrictions2) (/gemini-api/terms#use-restrictions2) apply to the grounded results.

What's next

- Learn more about the [Interactions API](/gemini-api/docs/interactions) (/gemini-api/docs/interactions).
- Learn how to use your own data using the [File Search](/gemini-api/docs/interactions/file-search) (/gemini-api/docs/interactions/file-search) tool.

Except as otherwise noted, the content of this page is licensed under the [Creative Commons Attribution 4.0 License](https://creativecommons.org/licenses/by/4.0/) (https://creativecommons.org/licenses/by/4.0/), and code samples are licensed under the [Apache 2.0 License](https://www.apache.org/licenses/LICENSE-2.0) (https://www.apache.org/licenses/LICENSE-2.0). For details, see the [Google Developers Site Policies](https://developers.google.com/site-policies) (https://developers.google.com/site-policies). Java is a registered trademark of Oracle and/or its affiliates.

Last updated 2026-05-29 UTC.